MultiSportPredict — Simple Command Guide & Cheat Sheet

START HERE: DATA INGESTION DIRECTORY • Copy complete command blocks into Git Bash.

📑 TABLE OF CONTENTS (CLICK TO JUMP)

1. The 3 Golden Rules
2. 🔄 DATA INGESTION DIRECTORY (ALL SPORTS)
3. 🎾 TENNIS
4. ⚽ SOCCER
5. ⚾ BASEBALL (MLB & KBO)
6. 🏈 FOOTBALL (NFL & NCAAF)
7. 🏀 BASKETBALL (NBA & International)
8. 📊 GRADING BETS & CHECKING WIN RATE
9. 💾 SAVING YOUR CODE TO GITHUB

THE 3 GOLDEN RULES

Open Git Bash from Windows Start, or VS Code → Terminal → New Terminal → Git Bash. Paste commands into the terminal. Begin every new session with:

```
cd /c/MultiSportPredict
source venv/Scripts/activate
pwd
venv/Scripts/python.exe --version
```

Use the project interpreter and -u for visible output. Commands below prompt for actual inputs; do not paste example teams, lines or odds as real data. Match prompts say whether the first team is home. A $ prompt means Bash is ready; a > prompt usually means an unfinished quote. Ctrl+C stops a running watcher.

This is the all-sports command reference. The only companion reference is the Czech Liga Pro guide: https://docs.google.com/document/d/1GFDeYLVTDHfpc4v5dgBB3f7gLOoCBm5mA1fJvG-yBtE/edit

Ingestion refreshes data. Predictions, grading and Discord are separate actions. Ingestion alone does not generate a prediction card. Check success/failure messages before running matches.

🔄 DATA INGESTION DIRECTORY (ALL SPORTS)

DATA INGESTION COMMANDS: choose the sport below. Run from /c/MultiSportPredict. These download/merge source data; blocked sources or empty responses are reported. Do not interpret a failed refresh as fresh data.

Czech Liga Pro table tennis: FREE ingestion

```
venv/Scripts/python.exe -u ingest_table_tennis_public.py --recent --max-events 10 --auto-train
venv/Scripts/python.exe -u ingest_table_tennis.py --stats
```

No API token needed. Collects visible finished matches with verified per-game points, exports CSV and attempts training. It does not fetch live sportsbook odds or automatically map Sofascore IDs to MyStake. Continuous collection:

```
venv/Scripts/python.exe -u ingest_table_tennis_public.py --recent --max-events 100 --watch --interval 1800 --auto-train
```

Leave the terminal open; Ctrl+C stops. For full match/grading instructions use the companion Czech guide linked above.

Tennis ATP / WTA: ingest and refresh history

```
MS_YEAR=$(date +%Y)
MS_PREV_YEAR=$((MS_YEAR - 1))
venv/Scripts/python.exe -u ingest_tennis.py --tours atp wta --years "$MS_PREV_YEAR" "$MS_YEAR"
```

Updates match/player history used by Elo. Failed tours retain previous history and the refresh is incomplete. Inspect failures; no source is guaranteed reachable.

Baseball: MLB and KBO

```
venv/Scripts/python.exe -u ingest_all_sports.py --only mlb mlb-probables mlb-lineups mlb-players
venv/Scripts/python.exe -u ingest_all_sports.py --only kbo
```

MLB players/props ingestion has its own source dependencies and can be slow. Failure of one adapter must be reviewed.

Football: NFL and NCAAF

```
MS_YEAR=$(date +%Y)
MS_PREV_YEAR=$((MS_YEAR - 1))
venv/Scripts/python.exe -u ingest_nfl.py --season "$MS_YEAR"
venv/Scripts/python.exe -u ingest_nfl_schedule.py --season "$MS_YEAR"
NCAAF_DATES=()
for d in 7 6 5 4 3 2 1 0; do NCAAF_DATES+=("$(date -d "$d days ago" +%Y-%m-%d)"); done
venv/Scripts/python.exe -u ingest_ncaaf.py --dates "${NCAAF_DATES[@]}"
venv/Scripts/python.exe -u ingest_ncaaf_model.py --seasons "$MS_PREV_YEAR" "$MS_YEAR"
```

NCAAF --dates accepts individual dates, not a start/end range. The loop supplies each of the past eight dates. The model ingestion builds opponent-adjusted score ratings separately from the recent-result collector.

Soccer: European and global leagues

```
venv/Scripts/python.exe -u ingest_soccer_fd.py --countries england germany spain france italy
venv/Scripts/python.exe -u ingest_soccer_espn.py --leagues epl la_liga bundesliga ligue_1 serie_a --fixtures
```

For all supported ESPN leagues, including Liga MX, MLS and others:

```
venv/Scripts/python.exe -u ingest_soccer_espn.py --all --fixtures
venv/Scripts/python.exe ingest_soccer_espn.py --list
venv/Scripts/python.exe ingest_soccer_fd.py --list
```

ESPN goals-derived xG estimates are tagged as estimates. The listed keys use underscores: la_liga, ligue_1, serie_a.

Basketball: NBA, WNBA and college

```
MS_YEAR=$(date +%Y)
MS_PREV_YEAR=$((MS_YEAR - 1))
MS_NEXT_YEAR=$((MS_YEAR + 1))
venv/Scripts/python.exe -u ingest_basketball_espn.py --leagues NBA WNBA NCAAB NCAAW --seasons "$MS_PREV_YEAR" "$MS_YEAR" "$MS_NEXT_YEAR"
```

Basketball: EuroLeague, KBL, NZ NBL and German BBL

```
venv/Scripts/python.exe -u ingest_all_sports.py --only euroleague
venv/Scripts/python.exe -u ingest_basketball_official.py --leagues KBL NZNBL BBL
```

KBL, NZNBL and BBL use the official source adapters. The aggregate alternative is ingest_all_sports.py --only kbl nznbl bbl. Official sources can still fail; the failure output identifies missing coverage.

Legacy master refresh and its missing steps

```
bash ingest_all.sh
```

This prints progress. The existing script covers MLB metrics/probables, KBO, ESPN soccer, ATP/WTA, NFL and recent NCAAF results. It does not include table tennis, all basketball ingestion or NCAAF model training. Add these separate commands after it:

```
venv/Scripts/python.exe -u ingest_ncaaf_model.py
venv/Scripts/python.exe -u ingest_basketball_espn.py --leagues NBA WNBA NCAAB NCAAW
venv/Scripts/python.exe -u ingest_all_sports.py --only euroleague
venv/Scripts/python.exe -u ingest_basketball_official.py --leagues KBL NZNBL BBL
venv/Scripts/python.exe -u ingest_table_tennis_public.py --recent --max-events 100 --auto-train
```

To save the legacy refresh output while displaying it:

```
set -o pipefail
bash ingest_all.sh 2>&1 | tee ingest_all.log
```

The legacy script can report [FAILED] without propagating every source failure through its exit status. Read its final summary and failure lines.

Source checks and data freshness

```
venv/Scripts/python.exe -u ingest_all_sports.py --list
venv/Scripts/python.exe -u ingest_all_sports.py --only kbl nznbl bbl --check
venv/Scripts/python.exe -u data_guard.py
```

--check avoids final store updates but some adapters may write diagnostic caches. Ingestion creates no Discord card; run a match and use that sport's separate push command below.

🎾 TENNIS (ATP & WTA)

Run a match with actual fixture properties

```
read -r -p "Match (First Player vs Second Player): " TEN_MATCH
read -r -p "Tournament: " TEN_TOURNAMENT
read -r -p "Surface (hard/clay/grass/carpet): " TEN_SURFACE
read -r -p "Round: " TEN_ROUND
read -r -p "Best of (3/5): " TEN_BEST
read -r -p "Match date (YYYY-MM-DD): " TEN_DATE
venv/Scripts/python.exe -u run_tennis.py --match "$TEN_MATCH" --tournament
"$TEN_TOURNAMENT" --surface "$TEN_SURFACE" --round "$TEN_ROUND" --best-of "$TEN_BEST"
--game-date "$TEN_DATE"
```

The runner attempts matched sportsbook moneyline and supported alternative game/set markets. Actual source/book coverage determines availability. Missing prices do not create EV. Thin/stale player history may refuse the run.

Two matches from the same fixture context

```
read -r -p "Second match (First Player vs Second Player): " TEN_MATCH_2
venv/Scripts/python.exe -u run_tennis.py --match "$TEN_MATCH" --match "$TEN_MATCH_2"
--tournament "$TEN_TOURNAMENT" --surface "$TEN_SURFACE" --round "$TEN_ROUND" --best-of
"$TEN_BEST" --game-date "$TEN_DATE"
```

Use the preceding tournament/date variables only when both fixtures really share them; otherwise run separately.

Manually add a real alternative line and its actual prices

```
read -r -p "Market flag (games-total/sets-total/games-spread/sets-spread): "
TEN_MARKET
read -r -p "Actual sportsbook line: " TEN_LINE
read -r -p "Price A (American odds): " TEN_PRICE_A
read -r -p "Price B (American odds): " TEN_PRICE_B
venv/Scripts/python.exe -u run_tennis.py --match "$TEN_MATCH" --tournament
"$TEN_TOURNAMENT" --surface "$TEN_SURFACE" --round "$TEN_ROUND" --best-of "$TEN_BEST"
--game-date "$TEN_DATE" "--$TEN_MARKET" 1 "$TEN_LINE" "$TEN_PRICE_A" "$TEN_PRICE_B"
```

Each alternative flag takes MATCH_NUMBER LINE PRICE_A PRICE_B. For totals A=Over/B=Under; spreads apply the signed line to the first supplied player. Repeat flags for additional lines. Never use assumed odds.

View and push reviewed rows

```
read -r -p "Review row number: " TEN_ROW
venv/Scripts/python.exe -u run_tennis.py --push "$TEN_ROW"
```

For two selected review rows, use --push 1 2 after checking the latest review. Rows are selections in that review, not necessarily one row per match.

Use sourced player context, or calculate without saving

```
read -r -p "Player Context JSON Path: " TEN_CONTEXT
```

```
venv/Scripts/python.exe -u run_tennis.py --match "$TEN_MATCH" --tournament
"$TEN_TOURNAMENT" --surface "$TEN_SURFACE" --round "$TEN_ROUND" --best-of "$TEN_BEST"
--game-date "$TEN_DATE" --context "$TEN_CONTEXT"
```

Context is an existing JSON file with sourced observations and any explicitly chosen rating scenarios. Injury/travel effects are not automatically measured. For a calculation with no database writes and no odds lookup:

```
venv/Scripts/python.exe -u run_tennis.py --match "$TEN_MATCH" --tournament
"$TEN_TOURNAMENT" --surface "$TEN_SURFACE" --round "$TEN_ROUND" --best-of "$TEN_BEST"
--game-date "$TEN_DATE" --no-store --no-odds
```

Push selected rows from the latest review with run_tennis.py --push and the row number, as above. A no-store run does not register bets to grade.

Full research package and separate push

```
venv/Scripts/python.exe -u run_tennis_full.py --match "$TEN_MATCH" --refresh
venv/Scripts/python.exe -u run_tennis_full.py --push
```

The full package needs a verified source fixture/history. --push sends its latest captured research report; it does not take a table-tennis tt_... ID. The package does not register a bet for ordinary grading automatically.

Grade logged tennis predictions

```
venv/Scripts/python.exe -u grade_predictions.py --auto --sport tennis --report
venv/Scripts/python.exe -u grade_predictions.py --report --sport tennis --push-discord
```

⚽ SOCCER (All Leagues)

Run one match

```
read -r -p "Home Team: " SO_HOME
```

```
read -r -p "Away Team: " SO_AWAY
read -r -p "League: " SO_LEAGUE
read -r -p "Game date (YYYY-MM-DD): " SO_DATE
venv/Scripts/python.exe -u universal_runner.py --sport soccer --home "$SO_HOME" --away "$SO_AWAY" --league "$SO_LEAGUE" --game-date "$SO_DATE" --live-odds --store-to-db
```

Runs without Discord. Matched real quotes determine which markets can have EV. Fixture/stat/source coverage must be verified; unavailable markets/players are not fabricated.

Run and push the single match

```
venv/Scripts/python.exe -u universal_runner.py --sport soccer --home "$SO_HOME" --away "$SO_AWAY" --league "$SO_LEAGUE" --game-date "$SO_DATE" --live-odds --store-to-db --push-discord
```

This reruns and may refresh prices. To review then push captured results use the batch workflow.

Batch: add each actual match interactively

```
venv/Scripts/python.exe -c 'import json; from pathlib import Path; matches=[]; count=int(input("Number of matches: ")); assert count>0; [(matches.append({"home":input("Home Team: ").strip(),"away":input("Away Team: ").strip(),"league":input("League: ").strip()})) for _ in range(count)]; p=Path("slate_soccer_today.json"); p.write_text(json.dumps({"matches":matches},indent=2),encoding="utf-8"); print("Saved:",p)'
venv/Scripts/python.exe -u run_soccer_batch.py --slate slate_soccer_today.json
```

The batch resolves upcoming source fixtures; it does not accept a --match CLI flag. Inspect the review before pushing.

```
venv/Scripts/python.exe -u run_soccer_batch.py --review
venv/Scripts/python.exe -u run_soccer_batch.py --push-all
```

Or push selected row(s):

```
read -r -p "Review row number: " SO_ROW
venv/Scripts/python.exe -u run_soccer_batch.py --push "$SO_ROW"
```

Both batch push options use the saved review, not a new slate calculation. Desired first-half/full-game, team-goal, corner and scorer markets require actual source data; their availability is not guaranteed.

Grade soccer and push the record

```
venv/Scripts/python.exe -u grade_predictions.py --auto --sport soccer --report
venv/Scripts/python.exe -u grade_predictions.py --report --sport soccer --push-discord
```

⚾ BASEBALL (MLB & KBO)

Run one game without sending Discord

```
read -r -p "League (MLB/KBO): " BB_LEAGUE
read -r -p "Match (Home Team vs Away Team): " BB_MATCH
read -r -p "Game date (YYYY-MM-DD): " BB_DATE
```

```
venv/Scripts/python.exe -u run_mlb.py --league "$BB_LEAGUE" --match "$BB_MATCH" --date "$BB_DATE" --odds --no-discord
```

Run and send Discord

```
venv/Scripts/python.exe -u run_mlb.py --league "$BB_LEAGUE" --match "$BB_MATCH" --date "$BB_DATE" --odds
```

The MLB/KBO runner sends by default when configured. It has no --push flag. Omit --no-discord to send. This reruns the game and can fetch new prices.

Run today's league slate

```
venv/Scripts/python.exe -u run_mlb.py --league "$BB_LEAGUE" --today --odds --no-discord
```

Discord version:

```
venv/Scripts/python.exe -u run_mlb.py --league "$BB_LEAGUE" --today --odds
```

Grade and push records

```
read -r -p "Grading sport (baseball for MLB / kbo): " BB_SPORT
venv/Scripts/python.exe -u grade_predictions.py --auto --sport "$BB_SPORT" --report
venv/Scripts/python.exe -u grade_predictions.py --report --sport "$BB_SPORT" --push-discord
```

🏈 FOOTBALL (NFL & College Football)

NFL: run a matchup or week

```
read -r -p "Match (Home Team vs Away Team): " NFL_MATCH
venv/Scripts/python.exe -u run_nfl.py --match "$NFL_MATCH" --odds
```

The runner validates against its real schedule and saves a review. Full-week alternative:

```
read -r -p "NFL week number: " NFL_WEEK
read -r -p "NFL season year: " NFL_YEAR
venv/Scripts/python.exe -u run_nfl.py --week "$NFL_WEEK" --season "$NFL_YEAR" --odds
```

NFL: push a reviewed selection

```
read -r -p "Review row number: " NFL_ROW
venv/Scripts/python.exe -u run_nfl.py --push "$NFL_ROW"
```

NCAAF: run a matchup

```
read -r -p "Match (Home Team vs Away Team): " CF_MATCH
read -r -p "Actual game date (YYYY-MM-DD): " CF_DATE
venv/Scripts/python.exe -u run_ncaaf.py --match "$CF_MATCH" --date "$CF_DATE" --odds
```

Uses the fitted score model built by ingest_ncaaf_model.py. Verify fixtures, team history and actual paired prices. More than one matchup can use repeated --match flags.

NCAAF: push the saved review

```
read -r -p "Review row number: " CF_ROW
venv/Scripts/python.exe -u run_ncaaf.py --push "$CF_ROW"
```

Grade each football sport and push its record

```
read -r -p "Sport (nfl/ncaaf): " FB_SPORT
venv/Scripts/python.exe -u grade_predictions.py --auto --sport "$FB_SPORT" --report
venv/Scripts/python.exe -u grade_predictions.py --report --sport "$FB_SPORT"
--push-discord
```

🏀 BASKETBALL (NBA, EuroLeague & International)

NBA, WNBA, NCAAB, NCAAW, KBL, NZNBL or BBL

```
read -r -p "League (NBA/WNBA/NCAAB/NCAAW/KBL/NZNBL/BBL): " BK_LEAGUE
read -r -p "Match (Home Team vs Away Team): " BK_MATCH
read -r -p "Actual game date (YYYY-MM-DD): " BK_DATE
venv/Scripts/python.exe -u run_basketball.py --league "$BK_LEAGUE" --match "$BK_MATCH"
--date "$BK_DATE" --odds
```

This score-model path supports full-game moneyline, spread and total. It does not promise all first-quarter/first-half or player-prop markets. Verify the actual review; missing source markets remain missing.

Push the saved league review

```
read -r -p "Review row number: " BK_ROW
venv/Scripts/python.exe -u run_basketball.py --league "$BK_LEAGUE" --push "$BK_ROW"
```

EuroLeague / international metrics and period projections

```
read -r -p "League: " INT_LEAGUE
read -r -p "Home Team: " INT_HOME
read -r -p "Away Team: " INT_AWAY
read -r -p "Game date (YYYY-MM-DD): " INT_DATE
venv/Scripts/python.exe -u universal_runner.py --sport basketball --home "$INT_HOME"
--away "$INT_AWAY" --league "$INT_LEAGUE" --game-date "$INT_DATE" --live-odds
--store-to-db
```

This is a different metrics/segment path. Quarter/half lines and EV require recognized paired quotes. Projected period points alone do not mean those markets are priced. Rebound/assist/points props require verified player/roster inputs; do not infer missing props from a team projection.

Run and push international basketball

```
venv/Scripts/python.exe -u universal_runner.py --sport basketball --home "$INT_HOME"
--away "$INT_AWAY" --league "$INT_LEAGUE" --game-date "$INT_DATE" --live-odds
--store-to-db --push-discord
```

This reruns and may refresh prices; unlike the score-model review push it is not a read-only saved-row send.

Manual alternative totals/spreads in the metrics runner

```
read -r -p "Actual alternative total line: " BK_TOTAL
read -r -p "Over price (American odds): " BK_OVER
read -r -p "Under price (American odds): " BK_UNDER
venv/Scripts/python.exe -u euroleague_engine.py --league "$INT_LEAGUE" --home
"$INT_HOME" --away "$INT_AWAY" --game-date "$INT_DATE" --alt-total "$BK_TOTAL"
"$BK_OVER" "$BK_UNDER" --output-json output/basketball_review.json
```

This requires sourced stored team metrics. It does not automatically fetch prices. --alt-spread takes HOME_LINE HOME_ODDS AWAY_ODDS. Use real observed values.

```
read -r -p "Actual home alternative spread (signed): " BK_SPREAD

read -r -p "Home price (American odds): " BK_HOME_PRICE

read -r -p "Away price (American odds): " BK_AWAY_PRICE

venv/Scripts/python.exe -u euroleague_engine.py --league "$INT_LEAGUE" --home
"$INT_HOME" --away "$INT_AWAY" --game-date "$INT_DATE" --alt-spread "$BK_SPREAD"
"$BK_HOME_PRICE" "$BK_AWAY_PRICE" --output-json output/basketball_review.json
```

To run that same manually priced spread and send Discord:

```
venv/Scripts/python.exe -u euroleague_engine.py --league "$INT_LEAGUE" --home
"$INT_HOME" --away "$INT_AWAY" --game-date "$INT_DATE" --alt-spread "$BK_SPREAD"
"$BK_HOME_PRICE" "$BK_AWAY_PRICE" --push-discord
```

For the manual total, use its own separate push command:

```
venv/Scripts/python.exe -u euroleague_engine.py --league "$INT_LEAGUE" --home
"$INT_HOME" --away "$INT_AWAY" --game-date "$INT_DATE" --alt-total "$BK_TOTAL"
"$BK_OVER" "$BK_UNDER" --push-discord
```

These manual metrics commands generate projections/reports. Add --store-to-db to log a priced run for later grading.

Grade basketball and push the record

```
venv/Scripts/python.exe -u grade_predictions.py --auto --sport basketball --report
venv/Scripts/python.exe -u grade_predictions.py --report --sport basketball
--push-discord
```

📊 GRADING BETS & CHECKING WIN RATE

All logged ordinary-sport predictions

```
venv/Scripts/python.exe -u grade_predictions.py --pending
venv/Scripts/python.exe -u grade_predictions.py --auto --report
```

--pending also writes pending_results.csv. Automatic grading depends on final-score source coverage and settleable logged selections. Logged research output does not mean a real bet was placed.

One sport and its Discord record

```
read -r -p "Sport (baseball/kbo/nfl/ncaaf/soccer/tennis/basketball): " GR_SPORT
venv/Scripts/python.exe -u grade_predictions.py --pending --sport "$GR_SPORT"
venv/Scripts/python.exe -u grade_predictions.py --auto --sport "$GR_SPORT" --report
venv/Scripts/python.exe -u grade_predictions.py --report --sport "$GR_SPORT"
--push-discord
```

Manual CSV results

Open pending_results.csv, fill actual final scores while retaining fixture identifiers, save and run:

```
venv/Scripts/python.exe -u grade_predictions.py --manual pending_results.csv --sport
"$GR_SPORT" --report
```

Send the updated record with the preceding --report --push-discord command.

Czech Liga Pro has its own grader

```
venv/Scripts/python.exe -u grade_table_tennis.py
```

Requires exact stored event IDs and all legal final game points. Use the companion guide's manual MyStake result-update workflow when automatic source reconciliation is unavailable. A prediction push does not send a table-tennis grading summary.

Backtest report

```
venv/Scripts/python.exe -u backtest_report.py --sport "$GR_SPORT"
```

A report measures the stored sample; it does not prove future profitability or independent selections.

💾 SAVING YOUR CODE TO GITHUB

These are version-control commands, not data refresh or prediction commands. Review changes before selecting files to commit.

```
git status --short
git diff --stat
git diff
read -r -p "Exact file path to stage: " GIT_FILE
git add -- "$GIT_FILE"
git diff --cached
read -r -p "Commit description: " GIT_MESSAGE
git commit -m "$GIT_MESSAGE"
git push
```

git push uses the current branch's configured upstream. Do not force a branch or stage .env, credentials or unreviewed data. If no upstream exists, inspect the actual branch/remote and set the intended upstream explicitly. No key values or commands that print .env belong in this document.